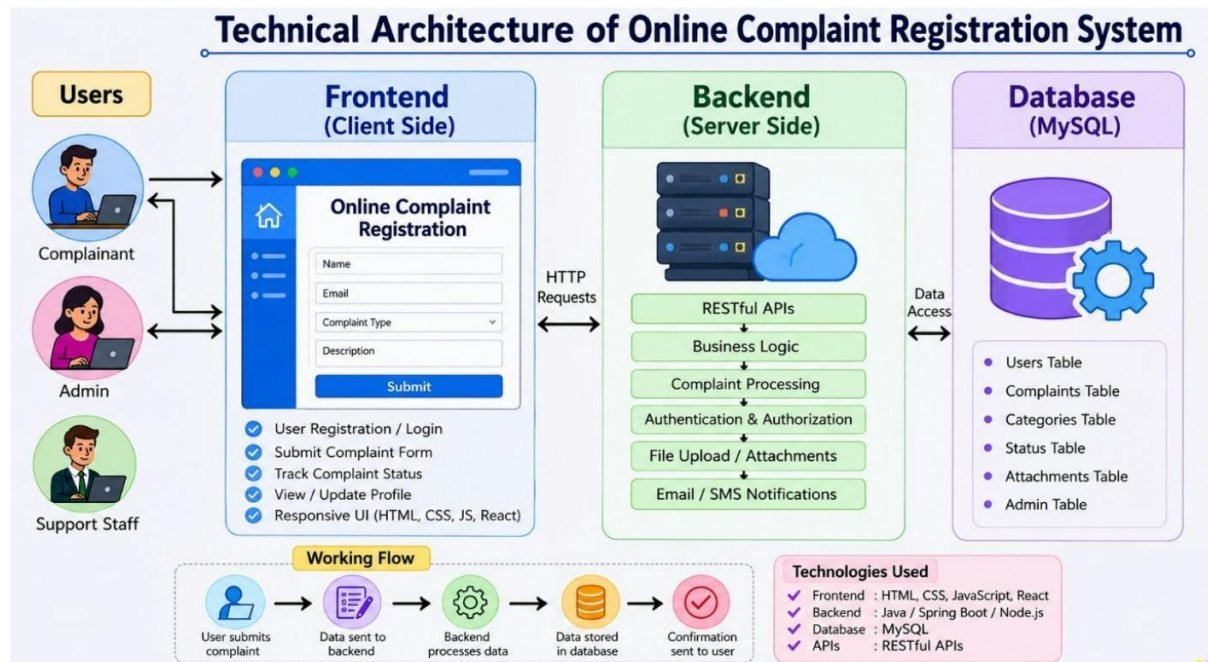


ONLINE COMPLAINT REGISTRATION AND MANAGEMENT SYSTEM

Project Documentation

Technology Stack: MongoDB · Express.js · React.js · Node.js (MERN)

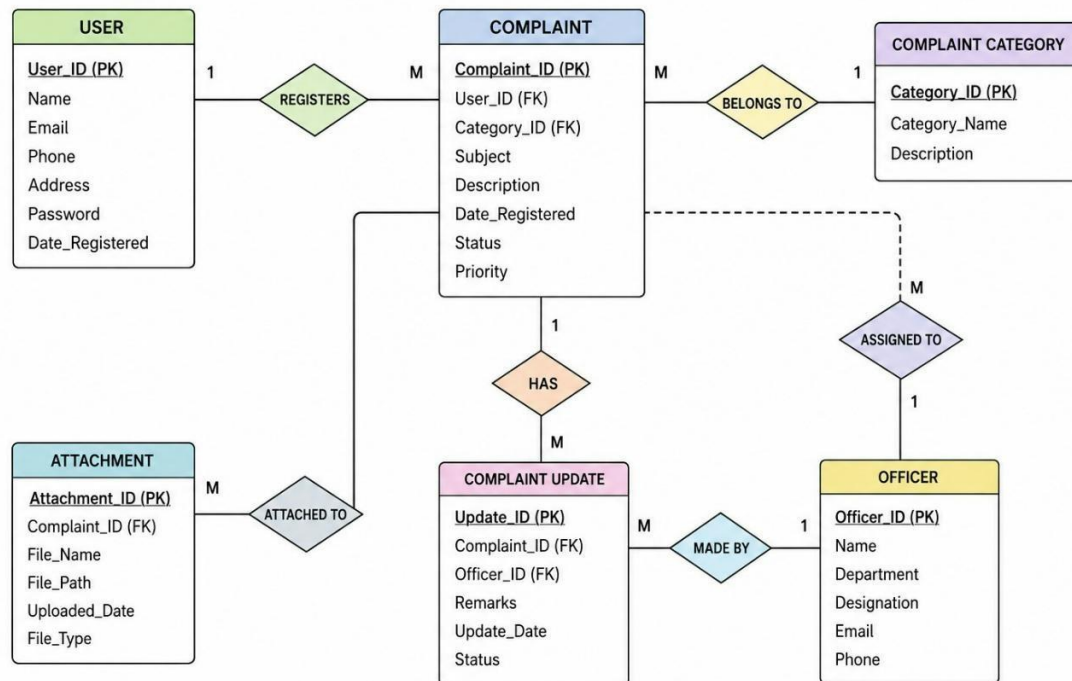
TECHNICAL ARCHITECTURE



The Technical Architecture of our **Online Complaint Registration System** follows a **client-server architecture**, where the frontend works as the client and the backend works as the server. The **frontend** provides a simple and user-friendly interface for users and admins to register complaints, view details, and track complaint status. It is developed using **HTML, CSS, JavaScript, and Bootstrap** and communicates with the backend using **REST APIs**. The **backend** handles the application logic, user authentication, complaint management, and data processing. It is developed using **Java with Spring Boot**. The system uses **MySQL database** to store user information, complaint details, and status updates securely. Together, the frontend, backend, APIs, and database provide a reliable platform for users to submit complaints and for administrators to manage and resolve them efficiently.

ER DIAGRAM

ER DIAGRAM FOR ONLINE COMPLAINT REGISTRATION SYSTEM



This ER diagram represents the Online Complaint Registration System. It shows the relationship between users, complaints, agents, and messages. Users can register and submit complaints by filling in the required details. Each complaint is assigned to an agent for review and resolution. The diagram also includes a chat feature that allows users and agents to communicate regarding a complaint. Messages are linked using the User ID and Complaint ID to ensure proper tracking of conversations. Overall, the ER diagram helps in understanding the database structure and the flow of information within the system.

FEATURES

1. User Registration

User Registration: Users can create an account by providing their email address and password to access the complaint management system.

Profile Management: Users can maintain their personal information, such as name, contact details, and address, for easy complaint tracking.

2. Complaint Categories

Category Selection: Users can choose the appropriate complaint category before submitting their complaint.

Department Filtering: The system helps users identify the relevant department responsible for handling their issue.

3. Complaint Submission

Online Complaint Submission: Users can submit complaints by providing details of the issue and uploading supporting documents or images.

Complaint Tracking: Users can monitor the status of their complaints from submission to resolution.

4. Officer Dashboard

Complaint Assignment: Officers can view and manage complaints assigned to their department.

Status Updates: Officers can update complaint progress, add remarks, and provide resolution details.

5. Admin Management

Officer Approval: Administrators verify and approve officer accounts to ensure authorized access.

System Monitoring: Admins oversee complaint activities, manage users, and generate reports for effective system management.

6. Notifications & Communication

Automated Notifications: Users receive updates regarding complaint submission, status changes, and resolution.

User–Officer Communication: The system enables communication between users and officers for clarifications and complaint follow-up.

ROLES AND RESPONSIBILITIES

1. User Registration

Example:

Priya wants to report a streetlight that is not working in her area. She visits the Online Complaint Registration System and creates an account by entering her name, email address, phone number, and password. After successful registration, she logs into the system.

2. Selecting Department & Complaint Category

Example:

Priya navigates through the available departments such as Electricity, Roads, Water Supply, and Sanitation. Since her issue relates to a faulty streetlight, she selects the Electricity Department and chooses the Streetlight Maintenance category.

3. Submitting a Complaint

Example:

Priya fills out the complaint form by describing the problem, providing the location details, and uploading a photo of the damaged streetlight. She then submits the complaint through the portal.

4. Complaint Acknowledgement

Example:

After submission, the system generates a unique complaint ID and sends a confirmation notification to Priya via email. The complaint is forwarded to the concerned department for review.

5. Complaint Status Tracking

Example:

Using her dashboard, Priya can monitor the complaint status, view updates from officers, and check whether the complaint is pending, under process, or resolved.

5. Admin Verification

Example:

The system administrator verifies officer accounts and department access rights. This process ensures that only authorized personnel can handle complaints and update records.

6. System Monitoring

Example:

The administrator monitors complaint activities, generates reports, manages escalated cases, and ensures the smooth operation and security of the platform.

7. Officer Complaint Handling

Example:

An officer from the Electricity Department reviews Priya's complaint, assigns it to a maintenance team, and updates the complaint status with remarks regarding the repair process.

8. Issue Resolution

Example:

The maintenance team repairs the streetlight and submits proof of completion. The officer marks the complaint as resolved, and Priya receives a notification confirming that the issue has been successfully addressed.

USER FLOW

User (Citizen)

Role: Register complaints, track complaint progress, communicate with officers, and manage account details.

Flow:

Account Creation and Login

- Register using email, phone number, and password.
- Log in securely to access the complaint portal.

Complaint Registration

- Select the appropriate department and complaint category.
- Enter complaint details and upload supporting documents or images.
- Submit the complaint and receive a unique complaint ID.

Complaint Monitoring

- View all submitted complaints from the dashboard.
- Track complaint status such as Submitted, Under Review, In Progress, and Resolved.

Communication

- Exchange messages with the assigned officer regarding complaint updates.
- Provide additional information when requested.

Profile Maintenance

- Update personal information and contact details.
- View complaint history and previous resolutions.

Officer

Role: Handle assigned complaints, communicate with users, and update complaint progress.

Flow:

Registration and Login

-  Create an officer account and log in using authorized credentials.

Complaint Handling

- Access assigned complaints through the officer dashboard.
- Review complaint details and supporting documents.

Complaint Processing

- Investigate issues and coordinate necessary actions.
- Add remarks, upload reports, and update complaint status.

User Communication

- Respond to user queries and provide progress updates.
- Request additional information when needed.

Administrator

Role: Manage the entire complaint management system, supervise officers, and ensure efficient complaint resolution.

Flow:

System Administration

- Monitor platform activities and complaint statistics.
- Ensure smooth functioning of the system. **Complaint**

Allocation

- Assign complaints to appropriate officers based on department and workload.
- Track pending and escalated complaints.

User and Officer Management

- Approve officer registrations and manage user accounts.
- Control access permissions and system roles.

Performance Monitoring

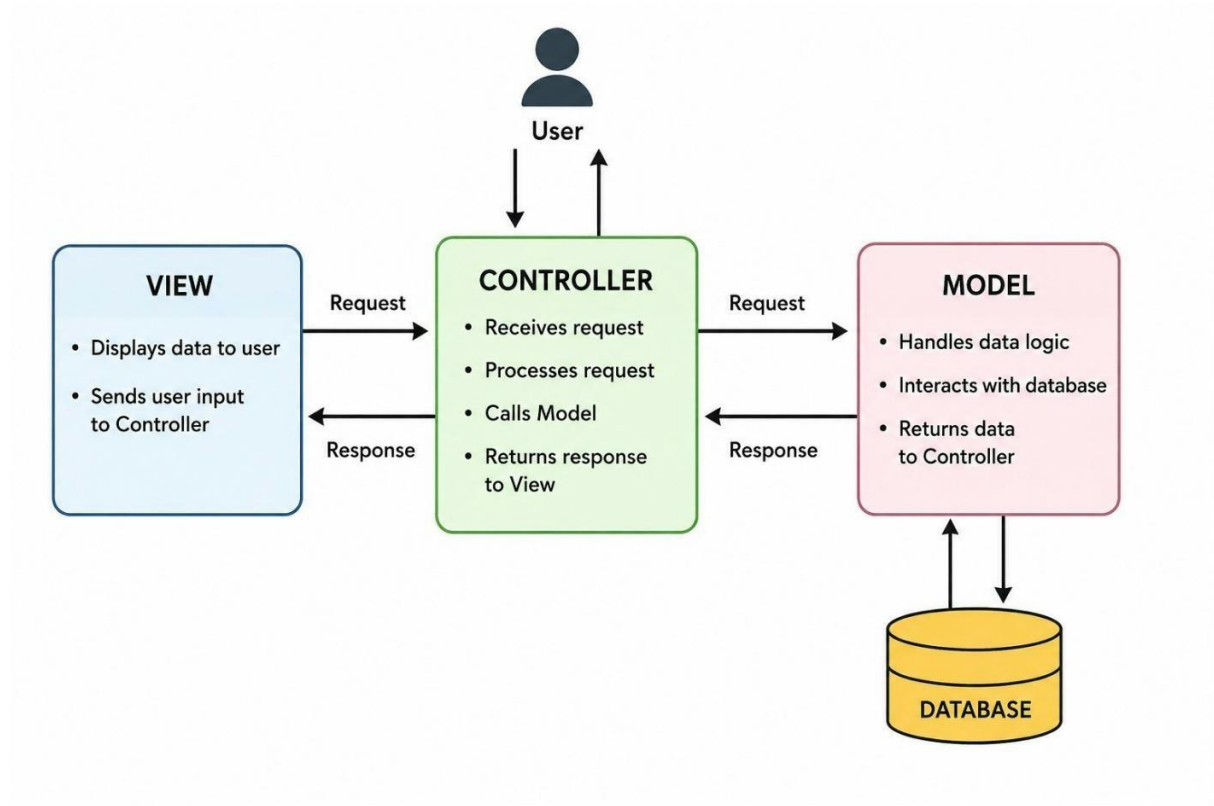
- Review complaint resolution rates and officer performance.
- Generate reports and analytics for decision-making.

Platform Maintenance

- Enforce security policies and data protection measures.
- Implement system improvements and resolve operational issues.

MVC Pattern

Description



MVC Architecture

The Online Complaint Registration System is developed using the **Model-View-Controller (MVC)** architecture. MVC divides the application into three independent components, making the system organized, maintainable, and easy to expand.

Model Layer (Data Management)

The **Model** layer manages all application data and database operations. It defines the structure of entities such as Users, Complaints, Officers, Messages, and Administrators. The model is

responsible for storing, retrieving, updating, and deleting information from the database while ensuring data consistency and integrity.

Controller Layer (Business Logic)

The **Controller** layer processes requests received from users and system administrators. It acts as a bridge between the View and the Model by handling business logic, validating user input, processing complaint submissions, managing complaint assignments, and updating complaint statuses. After processing the request, it returns the appropriate response to the user.

View Layer (User Interface)

The **View** layer represents the user-facing part of the application. It includes pages such as Registration, Login, Complaint Submission, Complaint Tracking, Officer Dashboard, and Admin Dashboard. The view collects user input and displays information received from the controller in a user friendly format.

Advantages of MVC Architecture in This Project

Modular

Design: Each component performs a specific function, making the application easier to understand and maintain.

Easy Maintenance: Changes in one layer have minimal impact on the other layers, reducing development complexity.

Scalability: Additional modules and features can be integrated without affecting the existing structure.

Code Reusability: Controllers and models can be reused across different modules of the system.

Improved Testing: Individual layers can be tested independently, improving software quality and reliability. **Team Collaboration:** Developers can work on the user interface, business logic, and database components simultaneously, increasing productivity.

CLIENT SETUP (installing react app)

Description

Setting Up the React Frontend

1. Open the **Client** folder in Visual Studio Code and launch the terminal.
2. Create a new React project using Vite by running the following command:

```
npm create vite@latest . -- --template react
```

3. When prompted, choose the following options:

○ **Framework:** React ○

Variant: JavaScript

4. Move to the client project directory:

```
cd client
```

5. Install the required dependencies:

```
npm install
```

6. Start the development server:

```
npm run dev
```

7. After the server starts successfully, open the URL displayed in the terminal (usually `http://localhost:5173`) to view the React application in your browser.

SERVER SETUP (npm init)

Backend Server Configuration

1. Open the **Server** folder in Visual Studio Code and launch the terminal.
2. Initialize a new Node.js project by executing the following command:

`npm init -y`

3. This command generates a **package.json** file with default project settings.
4. Create the main server file:

`server.js`

5. Organize the project structure by creating the following directories:

`controllers/`

`models/`

`routes/`

6. The purpose of each folder is:
 - **models** – Contains database schemas and data related logic.
 - **controllers** – Handles application logic and request processing.
 - **routes** – Defines API endpoints and connects them to controllers.
7. After creating the files and folders, the basic backend structure is ready for developing REST APIs and database integration.

DEVELOPMENT AND EXPLANATION

Project Setup

The development process begins by creating the server project and initializing it using the npm init command. Required packages such as Express.js, Mongoose, CORS, JWT, and other dependencies are installed to support backend functionality.

Express Server Creation

An Express.js server is configured to handle client requests and provide API services. Middleware such as JSON parsing and CORS is added to process request data and enable communication between the frontend and backend.

Route Development

Separate route files are created for different modules, including authentication, complaints, users, agents, and chat functionality. These routes define the available API endpoints and direct requests to the appropriate controllers.

Controller Implementation

Controllers are developed to handle application logic. They receive requests from routes, process the required operations, interact with database models, and send responses back to the client.

Database Model Creation

Mongoose

schemas and models are created for entities such as Users,

Complaints, Agents, Messages, and Admins. These models define the database structure and facilitate interaction with MongoDB.

Complaint Management

The complaint module allows users to create complaints, view complaint details, track status updates, and communicate with assigned agents. Agents can update complaint progress and provide resolutions.

User Authentication

Authentication is implemented using JSON Web Tokens (JWT). Users and agents can register and log in securely, while protected routes are accessible only to authorized users.

Admin Operations

Admin-specific features are developed to manage users, agents, and complaints. Administrators can assign complaints, monitor platform activities, and oversee complaint resolution processes. **Chat**

Functionality

A messaging system is implemented to enable communication between users and agents. Messages are stored in the database and linked to the relevant complaint for easy tracking.

Error Handling

Error-handling middleware is added to catch application errors and return meaningful error messages with appropriate HTTP status codes, improving system reliability and debugging.

System Testing

All modules are tested to ensure smooth functionality. APIs are verified, database operations are validated, and authentication and complaint workflows are checked for accuracy and performance.

CONFIGURE MONGODB

Install Mongoose

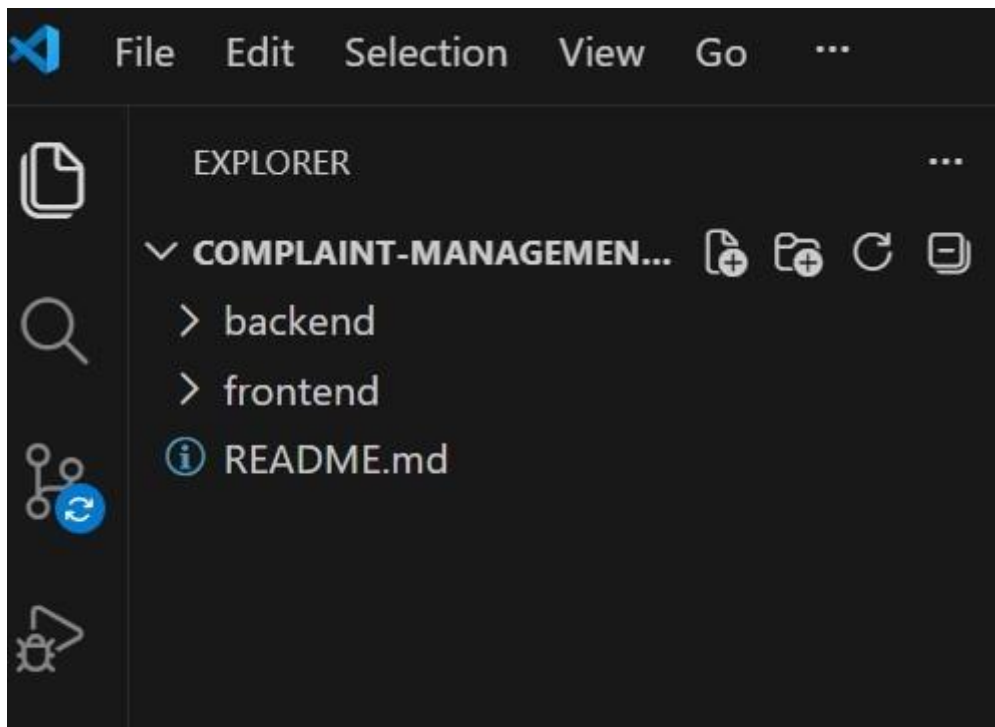
Mongoose is used to connect the Node.js application with the MongoDB database and perform database operations.

Run the following command in the terminal:

```
❏ npm install mongoose
```

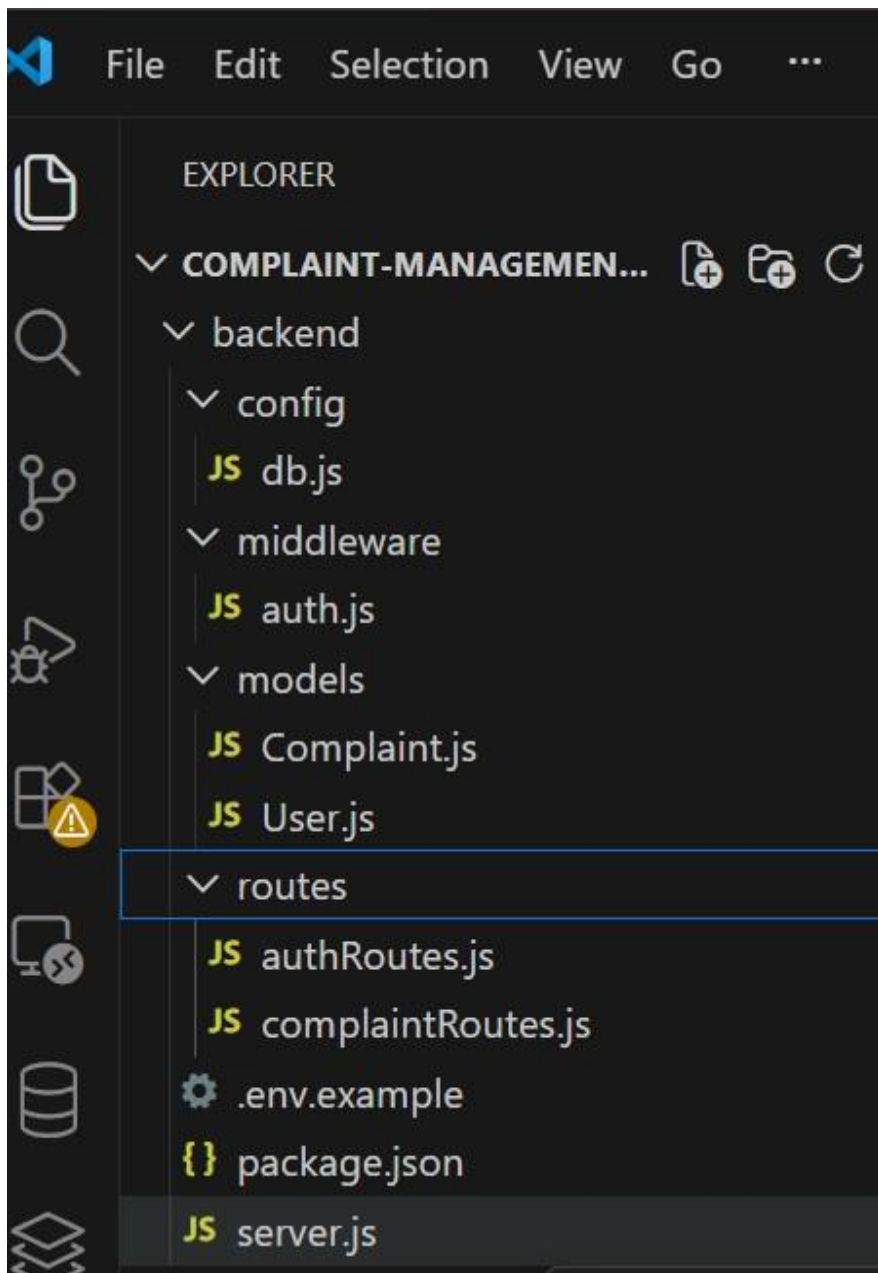
After installation, Mongoose can be used to create database schemas, models, and connect the application to MongoDB.

CREATING FOLDER STRUCTURE



The project is organized into a well-structured directory layout that separates the frontend and backend components, making the application easier to develop, maintain, and scale. The **backend** folder contains the server-side logic, including APIs, database models, routes, middleware, and configuration files. The **frontend** folder includes the user interface built with React, containing components, pages, assets, and styling files. This separation of concerns improves code readability and enables independent development and deployment of both modules. The **README.md** file provides project information, installation steps, and usage instructions for developers. Overall, this organized folder structure enhances maintainability, collaboration, and efficient project management.

BACKEND FOLDER STRUCTURE



The **Backend** folder contains the complete **server side implementation** of the Complaint Management System and is responsible for processing client requests, managing business logic, and communicating with the database. The **config** directory includes the **db.js** file, which establishes a secure connection with the **MongoDB** database. The **middleware** folder contains the

auth.js file, which implements **JWT (JSON Web Token) Authentication** to ensure secure access to protected resources. The **models** directory defines the **User** and **Complaint** schemas using **Mongoose**, providing a structured representation of data stored in the database. The **routes** folder manages all **RESTful API Endpoints**, including **User Authentication** and **Complaint Management** functionalities, ensuring organized request handling. The **server.js** file acts as the **main entry point** of the application by initializing the **Express.js Server**, configuring middleware, and registering API routes. Additionally, the **package.json** file manages project **dependencies, scripts**, and package information, while the **.env.example** file provides a template for **Environment Variables**, enhancing application **security, configuration management**, and **deployment flexibility**.

CREATE DATABASE CONNECTION

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const uri = process.env.MONGO_URI || "mongodb://localhost:27017/complaint_management";
    await mongoose.connect(uri);
    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

SCHEMA AND MODELS

```
const mongoose = require("mongoose");

const complaintSchema = new mongoose.Schema(
{
  title: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  category: {
    type: String,
    enum: ["Billing", "Technical", "Service", "Product", "Other"],
    default: "Other",
  },
  priority: {
    type: String,
    enum: ["Low", "Medium", "High", "Urgent"],
    default: "Medium",
  },
  status: {
    type: String,
    enum: ["Open", "In Progress", "Resolved", "Closed", "Rejected"],
    default: "Open",
  },
  complainant: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  assignedAgent: { type: mongoose.Schema.Types.ObjectId, ref: "User", default: null },
  attachments: [{ type: String }],
  comments: [
    {
      author: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
      message: String,
      createdAt: { type: Date, default: Date.now },
    },
  ],
  resolutionNote: { type: String, default: "" },
},
{ timestamps: true }
);

module.exports = mongoose.model("Complaint", complaintSchema);
```

```
const mongoose = require("mongoose");

const assignedComplaintSchema = mongoose.Schema({
  agentId: { type: mongoose.Schema.Types.ObjectId, required: true, ref: "User" },
  complaintId: { type: mongoose.Schema.Types.ObjectId, required: true, ref: "Complaint" },
  status: { type: String, required: true },
  agentName: { type: String, required: true },
});

module.exports = mongoose.model("AssignedComplaint", assignedComplaintSchema);
```

```

const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    password: { type: String, required: true },
    phone: { type: String, trim: true },
    role: { type: String, enum: ["user", "agent", "admin"], default: "user" },
  },
  { timestamps: true }
);

userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model("User", userSchema);

```

```

const mongoose = require("mongoose");

const messageSchema = mongoose.Schema({
  name: { type: String, required: 'Name is required' },
  message: { type: String, required: 'Message is required' },
  complaintId: { type: mongoose.Schema.Types.ObjectId, ref: "AssignedComplaint" }
}, { timestamps: true });

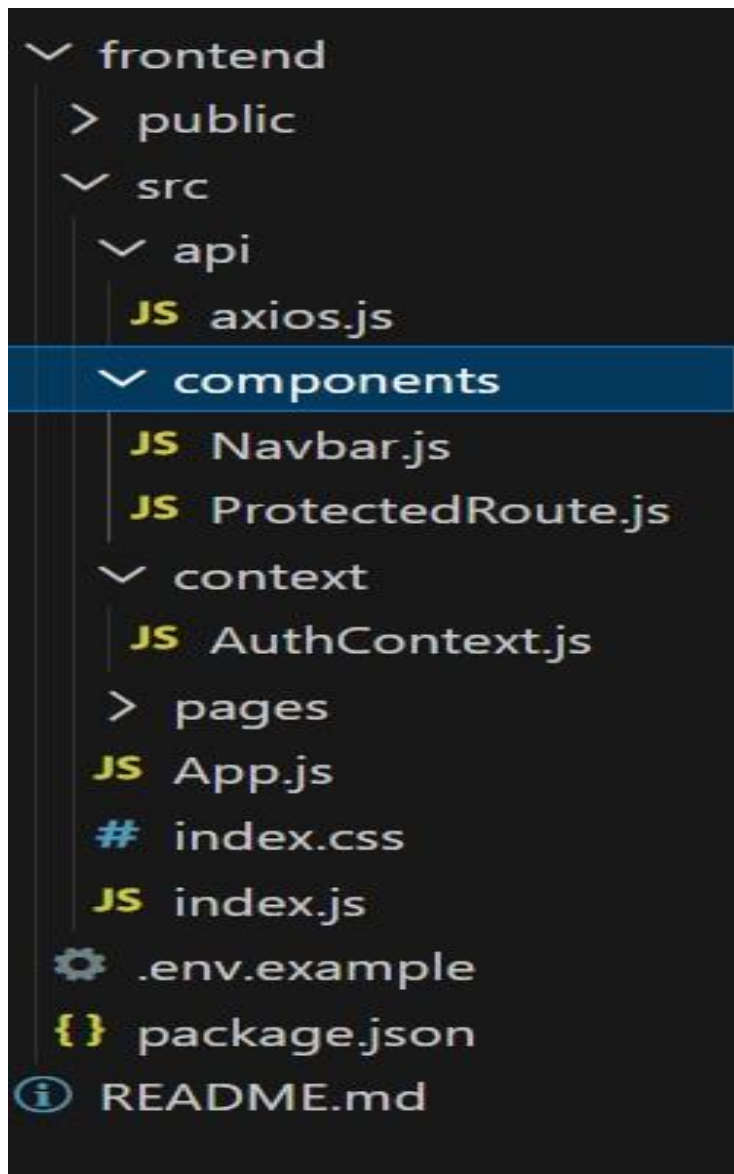
module.exports = mongoose.model("Message", messageSchema);

```

- Imports **Mongoose**, which provides schema-based modeling and convenient MongoDB
- operations in [Node.js](#).
- Declares a function called connectToDB. This will be called in your main server file (server.js or app.js) to initiate the DB connection.

- Connects to MongoDB using the **connection URI** stored in `process.env.MONGO_DB`.
- The second argument is an **options object**:
- `useNewUrlParser`: true ensures the new MongoDB connection string parser is used.
- `useUnifiedTopology`: true uses the new Server Discovery and Monitoring engine, which handles replica sets and sharding better.
- If the connection is **successful**, it logs a confirmation message.
- If the connection **fails**, it throws an error with a message.
- It's a good practice to **handle this error gracefully** instead of just throwing it. You could `console.error()` and exit the process using `process.exit(1)` if needed.
- Exports the `connectToDB` function so it can be used in other files, like `your server.js`:

FRONTEND FOLDER STRUCTURE



1.Setup React Application

The **frontend** folder contains the complete **client side implementation** of the Complaint

Management System, developed using **React.js**. The application structure is organized inside the **src** directory to maintain clean, reusable, and scalable code. The **api** folder contains **axios.js**, which is used to connect the frontend with backend **REST API endpoints**.

The **components** folder includes reusable UI elements such as **Navbar.js** for navigation and **ProtectedRoute.js** for securing private pages. The **context** folder contains **AuthContext.js**, which manages user authentication state across the application. The **pages** folder stores different screen level components such as login, register, dashboard, and complaint-related pages. The **App.js** file handles application routing, while **index.js** renders the React application into the browser. The **index.css** file manages global styling, and **package.json** maintains required frontend dependencies and scripts.

2.Design UI Components

The **UI components** are designed using **React.js** to provide a clean, responsive, and user-friendly interface for the Complaint Management System. The **Navbar.js** component provides smooth navigation between different pages such as **Home**, **Login**, **Register**, **Dashboard**, and **Complaint Management**. The **ProtectedRoute.js** component ensures that only authenticated users can access secure pages. The application layout is structured through the **pages** folder, where each page represents a specific screen or feature. This component-based design improves **code reusability**, **maintainability**, and **consistent user experience** across the application.

3.Implement Frontend Logic

The frontend logic connects the user interface with the backend services using **Axios** through the **axios.js** file. It handles **API communication**, including user login, registration, complaint creation, complaint viewing, and complaint status tracking. The **AuthContext.js** file manages the complete **authentication state**, stores user details, and controls login/logout functionality across the application. The **App.js** file manages **routing** and connects public and protected pages.

PROJECT EXECUTION

Steps for Project Execution:

Step 1: Download the Project

```
cd App
```

Step 2: Install Frontend Dependencies

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Step 3: Install Backend Dependencies

```
cd ../backend
```

```
npm install
```

Step 4: Create a .env file with your MongoDB URI:

```
MONGO_URI=mongodb://localhost:27017/OnlineCompla      int  
Registration
```

DEMO SCREENSHOTS

Output ScreenShots :

- **Landing Page** : Displays a welcome message with options to login or register.
Serves as the entry point for all users (User, Admin, Agent).

Complaint Management System

LoginRegister

Online Complaint Registration and Management System

A user-friendly platform to submit, track, and resolve complaints efficiently. Register your complaint, monitor its progress in real time, and communicate directly with assigned agents until resolution — all in one centralized, secure system.

Get StartedLogin

Key Features

- Secure user registration and authentication (JWT-based)
- Submit complaints with category and priority
- Real-time status tracking (Open, In Progress, Resolved, Closed)
- Agent assignment and admin oversight
- Comment thread for complainant-agent communication

- **Login Page** : Shows a form with email and password fields.

On successful login, redirects user to their respective dashboard.

Login

Email

Password

Login

Don't have an account? [Register here](#)

- **Registration Page** : Provides a form for new users to sign up with basic details.
Shows a success message and redirects to the login page upon registration.

Create an Account

Full Name

Email

Phone

Password

Register

Already have an account? [Login here](#)

- **Common Dashboard For Complaint** : Users can submit new complaints and view their complaint history. Each complaint displays its status (Pending, Assigned, Resolved) and assigned agent.

Submit a New Complaint

Title

Description

Category

Other

Priority

Medium

Submit Complaint

My Complaints

Title	Category	Priority	Status	Created
Internet Connectivity Issue in Computer Lab	Technical	High	Open	8/7/2026

- All Statuses
- All Statuses
- Open
- In Progress
- Resolved
- Closed
- Rejected

Admin Dashboard : Shows complaints assigned to the agent with their details.

Admin can update the status of complaints (e.g., In Progress, Resolved).

All Complaints

All Statuses

Title	Category	Priority	Status	Complainant	Created	
Internet Connectivity Issue in Computer Lab	Technical	High	Open	Leela	7/8/2026	View
Mismatched	Product	Medium	Open	Siva Leela Bijjam	7/6/2026	View
Tally problem	Billing	Urgent	Resolved	Sasidhar	7/3/2026	View

Tally problem

Resolved

Category: Billing | Priority: Urgent

The credit and debit amount transfer tally problem

Submitted by Sasidhar on 7/3/2026, 7:59:58 AM

Update Status: Resolved

Delete Complaint

Comments / Updates

Sasidhar7/3/2026, 8:00:17 AM

Not completed yet

Sasidhar7/3/2026, 8:00:39 AM

What is wrong with this!

Add a comment...

Post Comment

DEMO LINKS

https://drive.google.com/drive/folders/1IkNXwpGQBzt8yEjwa_ZvOKtdmLVr7be